

数据结构课程设计

-----小组设计报告

1 课程设计目的

学习获取知识的方法；

提高发现问题、分析问题和解决实际问题的能力；

加强创新意识和创新精神；

加强团队的分工与合作；

掌握面向实际背景思考问题的方法。

2 任务完成情况

表1 任务完成情况表

任务序号	任务名称	完成情况
1	基本功能----站点信息输入与读取	√
2	基本功能----线路边信息输入与建图	√
3	基本功能----站点状态修改	√
4	基本功能----线路信息查询	√
5	基本功能----路径查询功能	√
6	基本功能----用户界面	√
7	进阶功能----时间权重路径规划	√
8	进阶功能----最少换乘路径规划	√
9	进阶功能----多条候选路径查询	√
10	进阶功能----站点状态存储与恢复	√
11	进阶功能----受影响区域分析	√
12	进阶功能----站点模糊搜索	√

3 设计报告

3.1、问题描述

近年来，随着城市轨道交通网络不断扩张，地铁线路数量、换乘站规模和客流复杂度持续上升，传统依赖人工经验的线路查询方式已经难以满足乘客高效出行和运营状态动态调整的需求。一个可用的地铁路径规划系统，不仅要能够完成基础的站点查询和线路查询，还应当能够根据实际运营状态快速给出合理的出行方案。

本项目以地铁线路网络为研究对象，使用数据结构课程中的图结构知识，将地铁站点抽象为图中的顶点，将站点之间的连接关系抽象为图中的边，并在此基础上实现路径规划与运营管理功能。系统读取 **Station.csv**、**Edge.csv** 等数据文件，支持最短时间路径查询、最少换乘路径查询、多条候选路径输出、站点状态管理和关闭站点影响分析等功能。

与简单静态查询系统不同，本项目进一步考虑了真实运营中的动态因素，例如站点关闭、站点恢复、批量状态变更等，从而使得系统既具有数据结构课程中的算法训练价值，也具有一定的工程实践意义。

3.2、基本功能

系统采用 C++ 语言开发，利用邻接表图结构实现地铁网络建模，主要包含以下功能：

1. 地铁数据的加载与建图

- 从 CSV 文件加载地铁站点信息；
- 从 CSV 文件加载站点连接关系；
- 构建邻接表图结构。

2. 支持多种路径规划算法

- 单条最短时间路径查询；
- 3 条最短时间候选路径查询；
- 单条最少换乘路径查询；
- 3 条最少换乘候选路径查询。

3. 支持运营管理及网络分析

- 通过 CSV 批量修改站点开启/关闭状态；
- 手工修改站点开启/关闭状态；
- 恢复站点初始状态；

- 显示当前关闭站点；
- 关闭站点直接影响区域分析。

4. 系统交互

- 控制台二级菜单交互；
- 支持站点名称关键字模糊查询；
- 输出路径总时间、换乘站、换乘次数和经过站点数；
- 输出指定线路的站点与换乘信息。

3.3、功能模块设计

1. 数据加载与建模模块

- 读取 `Station.csv`、`Edge.csv`；
- 构建站点集合与邻接表图结构；
- 建立站点 id 到数组下标的快速索引。

2. 最优路径规划模块

- `shortestTimePath`：规划单条最短时间路径，在时间相同时按换乘次数更少优先；
- `kShortestTimePaths`：规划前 3 条较优时间路径；
- `minTransferPath`：规划单条最少换乘路径，在换乘次数相同时按总时间更少优先；
- `kMinTransferPaths`：规划前 3 条较优换乘路径；
- 剪枝策略：避免路径回路、去除重复候选、禁止经过关闭站点。

3. 路径输出与结果封装模块

- 将前驱数组还原为完整路径；
- 生成路径站点序列、站名序列、线路序列；
- 统计总时间、换乘次数、经过站点数；
- 输出换乘站列表。

4. 线路站点信息/运营状态管理模块

- `batchUpdateStatusFromCsv`：基于 `update\_station\_status.csv` 批量更新站点状态；
- `manualUpdateStationStatus`：手工指定站点状态；
- `resetStationStatus`：基于 `Station\_init.csv` 恢复初始状态；

- ``showClosedStations``：显示当前关闭站点名单；
- ``showLineInfo``：显示指定线路上的站点及换乘信息。

5. 用户交互模块

- 通过主菜单和子菜单组织功能；
- 负责接收用户输入、检查非法输入；
- 支持站点关键词搜索和二次选择；
- 输出查询结果并维持多轮交互。

3.4、代码实现

3.4.1 项目结构搭建

项目采用模块化设计，主要包含以下文件：

模块	对应文件	功能说明
公共定义模块	<code>`common.h`</code>	定义公共常量、数据结构和通用工具函数
站点管理模块	<code>`station.h`</code> / <code>`station.cpp`</code>	加载站点 CSV、管理站点状态、查询站点信息
图结构模块	<code>`graph.h`</code> / <code>`graph.cpp`</code>	根据边数据构建地铁图
路径规划模块	<code>`pathfinder.h`</code> / <code>`pathfinder.cpp`</code>	实现最短时间、最少换乘、多路径与影响分析
菜单控制模块	<code>`menu.h`</code> / <code>`menu.cpp`</code>	实现控制台菜单交互与功能调度
主函数模块	<code>`main.cpp`</code>	初始化系统并启动菜单

当前项目文件结构如下：

```
teamproject/
├── common.h
├── main.cpp
├── station.h
├── station.cpp
├── graph.h
├── graph.cpp
├── pathfinder.h
├── pathfinder.cpp
├── menu.h
├── menu.cpp
├── README.md
└── data/
    ├── Station.csv
    ├── Station_init.csv
    └── Edge.csv
```

└─ update_station_status.csv

3.4.2 站点数据加载模块实现 (station.cpp)

3.4.2.1 station.h

station.h 中定义了 **StationManager** 类，负责站点数据的统一管理。其主要对外接口包括：

- ``loadFromCsv``：从站点 CSV 文件加载全部站点信息。
- ``saveToCsv``：将当前站点状态保存回文件。
- ``resetFromInitCsv``：从初始文件恢复站点信息。
- ``batchUpdateStatusFromCsv``：批量更新站点状态。
- ``getStationById``：根据站点 id 查找站点。
- ``setStationStatus`` / ``setStationOpen``：修改站点状态。
- ``getClosedStations``：获取当前关闭站点。
- ``getStationsByLine``：获取指定线路上的全部站点。
- ``searchStationByName``：根据关键字模糊搜索站点。
- ``getAllStations``：获取所有站点。
- ``hasStation``：判断某个站点是否存在。

该类内部使用 `vector<Station>` 存储全部站点数据，并使用 `unordered_map<int, int>` 建立 id 索引，提高按编号查询的效率。

3.4.2.2 parseStatusText 和 toStatusText

parseStatusText 的作用是把 CSV 文件中的“开启”“关闭”文本转换为程序中的枚举类型 **StationStatus**。

toStatusText 的作用则相反，用于把程序中的枚举值转换为文本，以便保存回 CSV 文件。

这样设计的优点在于：

- 将内部逻辑和外部文件表示解耦；
- 避免业务代码中大量直接判断字符串；
- 提高代码可读性和可维护性。

3.4.2.3 rebuildIndex

rebuildIndex 用于在站点数据重新加载后，重新建立 **idToIndex_** 映射关系。

由于系统中很多操作都是通过站点 **id** 查找站点对象，因此索引表必须始终和 **stations_** 中的数据保持一致。

3.4.2.4 loadFromCsv

loadFromCsv 用于从 **Station.csv** 文件中逐行读取站点数据，并解析出：

- 站点 **id**
- 站点名称
- 所属线路
- 当前状态

解析完成后，将每一行数据构造成一个 **Station** 对象存入 **stations_** 容器中，最后调用 **rebuildIndex** 建立索引表。

该函数是整个系统完成初始化的基础。

3.4.2.5 saveToCsv

saveToCsv 用于将当前内存中的全部站点信息重新写回到 CSV 文件中。

当系统执行手工修改状态、批量更新状态等操作后，都需要调用该函数进行持久化保存。

3.4.2.6 resetFromInitCsv

resetFromInitCsv 通过读取 **Station_init.csv** 恢复初始站点信息，并将结果重新保存到当前使用的 **Station.csv** 中。

该函数使系统具备“恢复初始状态”的能力，便于调试、测试和重复演示。

3.4.2.7 batchUpdateStatusFromCsv

batchUpdateStatusFromCsv 用于读取 **update_station_status.csv** 中的更新指令，并批量修改站点状态。

其实现中包含以下处理：

- 校验每行格式是否正确；
- 校验 id 是否为合法整数；
- 校验状态是否为“开启”或“关闭”；
- 校验站点是否存在；
- 收集有效更新后统一执行。

这样可以降低因输入数据错误导致程序状态异常的风险。

3.4.2.8 getStationById

`getStationById` 提供了常量版本和可修改版本两个重载函数，分别用于：

- 只读查询站点信息；
- 在需要修改站点状态时直接取得站点对象。

由于内部采用 `idToIndex_` 映射，查找效率较高。

3.4.2.9 setStationStatus

`setStationStatus` 用于将指定站点状态直接设置为 `Open` 或 `Closed`。

这是底层状态修改函数，其他状态修改接口最终都会调用它。

3.4.2.10 setStationOpen

`setStationOpen` 使用布尔值来表示站点是否开启，便于菜单交互层直接调用。

例如输入 `1` 表示开启，输入 `0` 表示关闭。

3.4.2.11 isStationOpen

`isStationOpen` 用于判断某个站点当前是否处于开启状态。

该函数既可以用于路径规划前的合法性检查，也可以用于状态展示。

3.4.2.12 getClosedStations

`getClosedStations`

遍历全部站点，将状态为关闭的站点收集到结果列表中，供菜单模块统一显示。

3.4.2.13 getStationsByLine

getStationsByLine 根据线路名称筛选出属于该线路的所有站点。

该函数主要服务于“显示线路站点信息”功能。

3.4.2.14 searchStationByName

searchStationByName 使用子串匹配方式实现模糊查询。

只要用户输入的关键字出现在站点名称中，就将对应站点 `id` 加入结果集合。

该设计提高了站点查询和路径查询的便利性。

3.4.2.15 getAllStations

getAllStations 返回全部站点数据的只读引用。

图构建模块和菜单展示模块都需要依赖该函数获取完整站点列表。

3.4.2.16 hasStation

hasStation 用于判断某个站点 `id` 是否存在于系统中。

在建图、批量更新和路径查询前，这个判断都非常重要。

3.4.2.17 parseUpdateStatusText

parseUpdateStatusText 是定义在 `station.cpp` 匿名命名空间中的辅助函数，专门用于解析批量更新文件中的状态文本。

它只在当前实现文件内部使用，不暴露给其他模块。

3.4.3 图结构模块实现 (graph.cpp)

3.4.3.1 graph.h

graph.h 中定义了 **MetroGraph** 类，主要对外接口包括：

- ``buildFromCsv``：从边数据文件构建图；
- ``hasStation``：判断图中是否存在某站点；
- ``vertexCount``：返回图顶点数量；

- ``neighbors``: 返回某站点的邻接边集合。

类内部使用 `vector<vector<Edge>> adj_` 存储邻接表。

3.4.3.2 buildFromCsv

`buildFromCsv` 是图结构模块的核心函数。

它从 `Edge.csv` 中读取站点间连接关系，并将其构造成地铁网络图。

函数的主要处理流程如下：

1. 打开边数据文件；
2. 根据站点 `id` 范围初始化邻接表容量；
3. 逐行读取起点、终点、线路、方向和时间；
4. 校验边两端站点是否存在；
5. 构造 ``Edge`` 对象并加入邻接表。

此外，当边的线路名称为“换乘”时，系统会将该边标记为 `EdgeType::Transfer`，用于后续换乘次数统计。

3.4.3.3 hasStation

`hasStation` 用于判断图中是否具有指定编号的顶点。

该函数通过检查站点编号是否落在邻接表容量范围内实现。

3.4.3.4 neighbors

`neighbors` 返回指定站点的所有邻接边。

如果站点编号非法，则返回空向量，避免程序访问越界。

3.4.3.5 vertexCount

`vertexCount` 返回当前图中顶点容器的容量。

路径搜索时需要使用该值初始化距离数组和前驱数组。

3.4.3.6 countEdges

`countEdges` 用于统计图中边的总数。

虽然该函数当前没有直接对外业务调用，但它对于调试建图结果和核对数据规模很有帮助。

3.4.4 路径规划模块实现 (pathfinder.cpp)

3.4.4.1 pathfinder.h

`pathfinder.h` 中定义了 `PathFinder` 类。该类是本项目的算法核心，主要提供如下功能：

- ``shortestTimePath``：单条最短时间路径；
- ``minTransferPath``：单条最少换乘路径；
- ``kShortestTimePaths``：前 3 条较优时间路径；
- ``kMinTransferPaths``：前 3 条较优换乘路径；
- ``analyzeAffectedArea``：分析关闭站点造成的直接影响。

此外，类中还定义了若干辅助函数，如

`buildPathResult`、`countTransfers`、`isStationAvailable` 等，用于支持路径还原和可用性判断。

3.4.4.2 countsAsTransfer

`countsAsTransfer` 用于判断某条边是否应该计为一次换乘。

它的核心逻辑是：

- 如果该边是换乘边，则可能产生换乘；
- 但如果该边发生在起点站出发时，不额外记作一次换乘。

该规则使得换乘次数统计更符合实际理解。

3.4.4.3 makePathKey

`makePathKey` 用于把一条站点路径编码成一个字符串键值，例如把站点 `id` 按顺序拼接起来。

在多路径搜索中，该函数用于对结果路径去重，避免重复路径被多次输出。

3.4.4.4 containsStation

`containsStation` 用于判断某条候选路径中是否已经包含某个站点。

在扩展候选路径时，如果发现下一站已经在当前路径中，则不再继续扩展，从而避免回路出现。

3.4.4.5 countTransfers

countTransfers 用于统计一条路径中的换乘次数。

其做法是逐段检查相邻站点之间所使用的边，只要该边满足换乘条件，就将换乘次数加一。

3.4.4.6 buildPathResult

buildPathResult 根据最短路算法得到的前驱数组还原完整路径，并封装成 **PathResult** 结果对象。

该函数完成的工作包括：

- 反向回溯得到站点 id 序列；
- 还原站点名称与线路名称；
- 统计总时间；
- 提取换乘站名称；
- 填充路径是否有效、总时间、换乘次数和经过站点数。

该函数的存在使得算法层与显示层实现解耦。

3.4.4.7 shortestTimePath

shortestTimePath 用于查询单条最短时间路径。

其算法思路为：

- 采用优先队列进行最短路搜索；
- 以“总时间”为第一关键字；
- 当总时间相同时，以“换乘次数更少”为第二关键字。

该策略既保证了时间最优，也使结果在同等时间下更合理。

3.4.4.8 minTransferPath

minTransferPath 用于查询单条最少换乘路径。

与最短时间路径不同，它的优先级设计为：

- 第一关键字：换乘次数最少；
- 第二关键字：总时间最少。

这样可以满足部分乘客“宁可多坐一会，也希望少换乘”的出行偏好。

3.4.4.9 kShortestTimePaths

kShortestTimePaths 用于查询前 3 条时间较短的候选路径。

其核心实现思想包括：

- 用优先队列维护候选路径状态；
- 每次扩展当前最优路径；
- 使用 `containsStation` 防止路径回路；
- 使用 `makePathKey` 进行结果去重；
- 使用 `expandedCount` 和 `maxExpansions` 控制搜索规模。

该算法能够较好地给出多个不同的可行备选方案。

3.4.4.10 kMinTransferPaths

kMinTransferPaths 用于查询前 3 条换乘次数较少的候选路径。

其搜索框架与 **kShortestTimePaths** 基本一致，只是比较优先级改为“换乘优先，时间次优先”。

3.4.4.11 analyzeAffectedArea

analyzeAffectedArea 用于分析关闭站点周边站点的直接影响。

其处理思路为：

- 将关闭站点集合放入哈希集合；
- 遍历每个关闭站点的邻接边；
- 找到所有与关闭站点直接相连、但自身未关闭的站点；

- 去重并排序后返回。

该功能可以用于简单模拟地铁站关闭后的邻近受影响区域。

3.4.4.12 isStationAvailable

isStationAvailable 用于判断某站点是否可以参与路径搜索。

只有当站点存在、图中有对应顶点且状态为开启时，才允许进入路径规划。

3.4.5 菜单控制模块 (menu.cpp)

3.4.5.1 menu.h

menu.h 中定义了 **MenuSystem** 类，负责整个控制台交互流程的组织 and 调度。

该类把菜单显示、路径查询、状态管理和结果输出统一封装在一起。

3.4.5.2 MenuSystem

构造函数 `MenuSystem(StationManager&, MetroGraph&, Pathfinder&)`

接收三个核心模块的引用，使菜单模块能够直接调用数据管理、图结构和路径规划功能。

3.4.5.3 showMainMenu

showMainMenu 用于显示系统主菜单，包含：

- 线路站点信息/运营状态管理
- 所需时间最短路径规划
- 所需换乘次数最少路径规划
- 退出系统

3.4.5.4 showStationMenu

showStationMenu 用于显示站点管理子菜单，包含批量更新、手工更新、查看关闭站点、恢复初始状态、线路查询、受影响分析和站点查询等功能。

3.4.5.5 showTimePathMenu

showTimePathMenu 用于显示最短时间路径查询菜单，支持：

- 单条最短时间路径
- 3 条最短时间路径

3.4.5.6 showTransferPathMenu

`showTransferPathMenu` 用于显示最少换乘路径查询菜单，支持：

- 单条最少换乘路径
- 3 条最少换乘路径

3.4.5.7 run

`run` 是菜单系统的主循环函数。

其作用是：

- 持续显示主菜单；
- 接收用户输入；
- 检测输入是否合法；
- 分发到不同的子功能模块；
- 直到用户主动选择退出。

3.4.5.8 handleStationMenu

`handleStationMenu`

负责站点管理菜单的循环处理，根据用户输入调用相应的站点管理功能。

3.4.5.9 handleTimePathMenu

`handleTimePathMenu` 负责最短时间路径菜单的循环处理。

3.4.5.10 handleTransferPathMenu

`handleTransferPathMenu` 负责最少换乘路径菜单的循环处理。

3.4.5.11 batchUpdateStationStatus

`batchUpdateStationStatus` 调用 `StationManager::batchUpdateStatusFromCsv` 批量更新站点状态，并在成功后调用 `saveToCsv` 保存修改结果。

3.4.5.12 `manualUpdateStationStatus`

`manualUpdateStationStatus` 根据用户输入的站点 `id` 和状态值，手工修改站点状态并保存到数据文件中。

3.4.5.13 `resetStationStatus`

`resetStationStatus` 调用 `resetFromInitCsv` 恢复所有站点的初始状态。

3.4.5.14 `showClosedStations`

`showClosedStations`

从站点管理模块取得当前全部关闭站点，并按照统一格式输出到控制台。

3.4.5.15 `lineNumberToName`

`lineNumberToName` 用于把用户输入的线路编号转换成系统内部使用的线路名称，例如 1 转换为 1号线，19 转换为 浦江线。

3.4.5.16 `showLineInfo`

`showLineInfo` 用于展示指定线路的全部站点，并识别每个站点是否具有换乘功能。

其实现方式是：

- 先根据线路名筛选该线路所有站点；
- 再遍历全部站点，查找“同名但不同线”的站点；
- 若存在，则将其视为换乘线路进行展示。

3.4.5.17 `analyzeAffectedStations`

`analyzeAffectedStations` 先通过关键词让用户选定一个站点，再调用 `PathFinder::analyzeAffectedArea` 输出其关闭后直接受影响的站点列表。

3.4.5.18 `searchStationInfo`

searchStationInfo 支持通过关键字搜索站点，并输出：

- 站点 id
- 站点名称
- 所属线路
- 当前状态

3.4.5.19 selectStationByKeyword

selectStationByKeyword 是多个查询功能共用的辅助函数。

它先根据关键字匹配所有候选站点，再显示候选列表，最后由用户输入编号选择具体站点。

这个设计可以解决模糊搜索结果不唯一的问题。

3.4.5.20 queryShortestTimePath

queryShortestTimePath

负责组织最短时间单路径查询流程，包括起点选择、终点选择、调用算法和输出结果。

3.4.5.21 queryKShortestTimePaths

queryKShortestTimePaths 负责组织最短时间多路径查询流程，并依次打印 3 条候选路径。

3.4.5.22 queryMinTransferPath

queryMinTransferPath 负责组织最少换乘单路径查询流程。

3.4.5.23 queryKMinTransferPaths

queryKMinTransferPaths 负责组织最少换乘多路径查询流程。

3.4.5.24 printPathResult

printPathResult 用于将路径结果格式化输出，内容包括：

- 预计车程
- 途经站点数
- 换乘次数

- 路径线路
- 站点路径
- 换乘站

3.4.5.25 waitForEnter

waitForEnter

用于在当前操作结束后暂停，等待用户按回车继续，便于分步演示和观察输出结果。

3.4.6 公共调用模块 (common.h)

3.4.6.1 common.h

common.h 是全项目的公共头文件，主要定义了以下内容：

- 常量：`INF`、`TRANSFER\_TIME`
- 枚举：`StationStatus`、`EdgeType`
- 结构体：`Station`、`Edge`、`PathResult`

这些定义为后续站点管理、图结构构建和路径规划提供了统一的数据基础。

3.4.6.2 便于和 csv 文件交互的函数

common.h 中定义了两个重要的内联工具函数：

1. `trimCsvField`

- 去除 CSV 字段前后的空白字符；
- 避免因格式问题造成读取错误。

2. `resolveDataPath`

- 自动查找数据文件路径；
- 支持当前目录和上层目录查找；
- 便于项目在不同运行目录下正确读取 `data` 文件夹中的文件。

resolveDataPath 内部还使用了若干 lambda 辅助函数：

- `isAbsolutePath`：判断是否为绝对路径；

- ``hasDirectoryPart``: 判断路径中是否已包含目录信息;
- ``joinPath``: 拼接路径;
- ``parentPath``: 获取父目录;
- ``fileExists``: 检查文件是否存在。

3.4.7 主函数 (main.cpp)

3.4.7.1 main

main 函数是整个程序的入口，其主要执行流程如下：

1. 设置控制台输入输出编码为 UTF-8;
2. 通过 ``resolveDataPath`` 定位 ``Station.csv`` 和 ``Edge.csv``;
3. 创建 ``StationManager`` 并加载站点数据;
4. 创建 ``MetroGraph`` 并构建线路图;
5. 创建 ``PathFinder``;
6. 创建 ``MenuSystem`` 并进入菜单主循环。

该函数将各模块串联起来，形成完整可运行系统。

3.4.7.2 最终项目文件结构

程序最终运行时由以下模块协同完成：

- ``main.cpp``: 负责初始化;
- ``station.cpp``: 负责站点信息加载与状态管理;
- ``graph.cpp``: 负责建图;
- ``pathfinder.cpp``: 负责核心算法;
- ``menu.cpp``: 负责用户交互。

整个程序遵循“先加载数据，再构建图，最后提供交互”的基本执行逻辑。

3.5、使用示例

3.5.1 系统主菜单

程序启动后，系统显示主菜单，用户可以在三类核心功能之间切换：

1. 线路站点信息/运营状态管理
2. 所需时间最短路径规划
3. 所需换乘次数最少路径规划
4. 退出系统

3.5.2 线路站点信息/运营状态管理

进入该菜单后，可以看到以下子功能：

1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 受关闭站点影响站点分析
7. 站点查询
8. 返回上级菜单

3.5.3 从 CSV 文件批量更新站点状态

当用户选择批量更新功能后，系统会自动读取 `update_station_status.csv` 中的更新记录，并修改 `Station.csv` 中对应站点的状态。

如果更新文件格式正确且存在有效记录，系统会输出“批量更新成功”；否则输出“批量更新失败”。

3.5.4 手工更新站点状态

用户输入站点 `id` 和状态值后，系统会立即修改内存中的站点状态，并保存到 `Station.csv` 中。

如果站点存在，则输出“站点状态更新成功”；若站点不存在，则输出“指定站点未找到”。

3.5.5 显示当前关闭站点

系统会列出当前所有关闭站点，并显示其站点名称和所属线路。

若当前不存在关闭站点，则提示“当前无关闭站点”。

3.5.6 恢复所有站点初始状态

用户选择该功能后，系统会以
为基础恢复全部站点状态，适合在测试后快速回到初始状态。

Station_init.csv

3.5.7 显示线路站点信息

用户输入线路编号后，系统会输出该线路上的全部站点。

对于具有换乘能力的站点，系统会额外标注可换乘的其他线路；如果该站当前处于关闭状态，也会同步标记。

3.5.8 受关闭站点影响站点分析

用户先输入一个站点关键字并选择具体站点，系统再把该站点视为关闭状态，输出所有直接受影响的相邻站点。

每个受影响站点会显示站点名称、所属线路和当前状态。

3.5.9 站点查询

用户输入站点名称关键字后，系统会先列出所有匹配站点，再由用户选择一个具体站点，最后展示其完整信息。

3.5.10 所需时间最短路径规划

用户进入该子菜单后，可以选择：

1. 单条所需时间最短路径
2. 3 条所需时间最短路径
3. 返回上级菜单

3.5.11 单条所需时间最短路径

系统先让用户通过关键字选择起点和终点，然后调用 `shortestTimePath` 进行查询。

输出结果中包括总时间、经过站数、换乘次数、路径线路、站点路径和换乘站。

3.5.12 3 条所需时间最短路径

系统调用 `kShortestTimePaths`，依次输出 3 条时间较短的候选路径。

该功能适合在最优路径不可行或用户希望进行方案比较时使用。

3.5.13 所需换乘次数最少路径规划

用户进入该子菜单后，可以选择：

1. 单条所需换乘次数最少路径
2. 3 条所需换乘次数最少路径
3. 返回上级菜单

3.5.14 单条所需换乘次数最少路径

系统调用

`minTransferPath`

查询换乘次数最少的路径，并在换乘次数相同的情况下优先输出总时间更短的路径。

3.5.15 3 条所需换乘次数最少路径

系统调用

`kMinTransferPaths`

输出

3

条换乘次数较少的备选路径，便于用户进行出行方案选择。

3.6、测试与调试

在测试和调试过程中，主要对以下内容进行了验证：

1. 数据读取测试
 - 检查 `Station.csv`` 和 `Edge.csv`` 是否能够正确读取；
 - 检查建图后顶点和边的数量是否符合预期。
2. 输入合法性测试
 - 测试主菜单和子菜单输入非数字时系统是否能恢复；
 - 测试输入越界编号时系统是否给出提示；
 - 测试站点关键字无匹配时系统是否能正确返回。
3. 路径正确性测试

- 测试普通情况下最短时间路径是否合理；
- 测试最少换乘路径是否满足“换乘优先”要求；
- 测试多条候选路径是否存在重复。

4. 状态管理测试

- 测试手工关闭站点后是否能影响路径查询；
- 测试批量更新后状态是否写回文件；
- 测试恢复初始状态后数据是否回到默认值。

5. 影响分析测试

- 测试关闭某个站点后，系统能否正确识别其直接相邻的受影响站点。

调试过程中也发现了一些需要注意的问题：

- CSV 文件路径在不同运行目录下可能失效，因此增加了 ``resolveDataPath`` 自动查找机制；
- 菜单输入如果不处理 ``cin.fail()``，容易导致输入流错误循环，因此在菜单模块中加入了输入恢复逻辑；
- 多路径搜索如果不做去重和防回路处理，容易产生重复路径或无意义扩展，因此加入了路径去重和扩展上限控制。

4 体会与感想

通过本次课程设计，我们把课堂中学到的图、优先队列、哈希表、字符串处理和文件读写等知识综合运用到了一个较完整的实际问题中。与单纯做算法题相比，这次项目更强调“如何把数据结构知识真正组织成一个能运行、能维护、能演示的系统”。

在实现过程中，我们体会最深的有以下几点：

1. 模块化设计非常重要。

如果没有将站点管理、图结构、路径规划和菜单交互拆分开，后续修改功能会非常混乱。

2. 数据结构的选择直接影响实现难度和运行效率。

例如邻接表适合表示地铁网络，`unordered_map` 适合按 id 快速定位站点，优先队列适合实现路径优化搜索。

3. 实际工程比单独写算法更复杂。

除了算法本身，还要考虑输入异常、文件路径、数据持久化、状态恢复和结果展示等问题。

4. 课程设计锻炼了完整的分析与实现能力。

从问题理解、模块划分、代码实现，到测试验证和文档整理，整个过程帮助我们更系统地理解了数据结构课程知识的实际应用。

总体来看，本项目较好完成了地铁路径规划系统的设计与实现任务，既体现了课程知识，也提升了团队协作和工程实践能力。